

NELSON MANDELA UNIVERSITY



Faculty of Science

WRPV302: Advanced Programming 3.2

Assignment 1: Due Tuesday, 11 August 2026

Department of Computing Sciences

Please read this document carefully and keep it in a safe place — it contains important information that you will need to refer to during the semester.

Possession of this document **does not** guarantee your registration (or acceptance) for this module, nor does it guarantee access to the departmental computer laboratories.

Contents

General assignment information	4
General assignment procedure	4
Assignment schedule	6
Assignment 1	7
Topics	7
Task 1: Android “Hello World”	7
Task 2: Android Sequence-Dice	7
Rubric	12

General assignment information

This document contains the information that is needed to complete Assignment 1 for the WRPV302 module.

Be sure to submit your assignment in a zipped file on the Funda site before the due date and time. Late submissions will not be accepted. You are also required to *peer* assess assignments submitted by your peers for all the practical assignments.

Remember to complete *all* the Activities, as *all* of them count towards the Final Mark for this Module.

Note: Practical Assignments are **not** group Assignments, they are to be completed **individually**. Be sure to read the Academic Integrity section in the Module Guide. The **minimum** penalty for any acts of Plagiarism or Cheating is 0% for that specific assignment for **all** parties involved.

Note: Working individually means you do the work *yourself*. Using other people's code is **not** individual work. Where an assignment explicitly permits or requires the use of an AI coding assistant, the rules stated in that task's instructions apply, and use must be declared as set out there. The five-line limit applies to code taken from other people's work, not to AI-assisted code produced under a declared protocol. When AI is not permitted, making use of code generated by ChatGPT (or anything else) is **not** individual work – *you* did not write the code. If detected, the minimum punishment is zero for the assignment for all involved (i.e., the one(s) copied from *and* the one(s) doing the copying).

Note: You **are** allowed to "plagiarise" lecture example code in assignments, i.e., you may extend/modify lecture example code to your heart's delight. A **maximum** of 5 lines¹ of code from *another* source for an *entire* task in an assignment is allowed. These 5 lines must be clearly marked as coming from elsewhere and a reference to the code must be provided.

Note: At 3rd year level, **all** programs are expected to compile and run. If a task does not compile without syntax errors, it will be given a zero!

General assignment procedure

Please read this entire section before doing anything:

- Assignments are to be completed and submitted before the cut-off date. Submission is via the module's Funda page (<https://funda.mandela.ac.za/>).
- **Late submissions are not allowed and will result in an automatic mark of 0!** Do *not* attempt submission 5 minutes before the cut-off time – *anything* can go wrong. Submit timeously.
- **Submit your solutions in a single zip file.** Please use proper naming conventions, document your code using comments and write neat, easy-to-read code. **If using Android Studio, IntelliJ and Java at home, *please* use the same versions and install them in the same locations as on the lab machines.** This makes transferring projects between computers easier, and it will make your assessors' lives easier if they do not need to struggle to get your projects to compile and run on their machines. This will probably result in better marks for you.

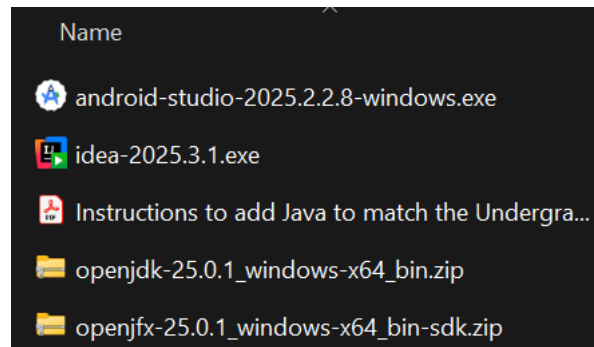
¹ Five lines of properly formatted code, i.e., 5 lines of code after the auto-format tool is used.

For this academic year, the following software versions are being used (**i.e., use these versions at home, even if newer versions exist**):

- Android Studio 2025.2.2.8 (the About window displays this as *Android Studio Otter 2 Feature Drop | 2025.2.2 Patch 1*)
- IntelliJ 2025.3.1
- OpenJDK 25.0.1
- OpenJavaFX SDK 25.0.1

You may find the installation files and some instructions on the courses folder

[\\postgrad-new.mandela.ac.za\wrpv software](https://postgrad-new.mandela.ac.za/wrpv-software):



Be sure to connect to the VPN to access the folder above.

On a weekly basis, you will be required to:

- *self-assess* your submission. This process will involve “marking” the previous assignment’s solution that you submitted. Its intention is to have you critically evaluate your own work according to the rubric provided on the Funda site; and
- *peer assess* some of your peers (as well as your own work). This process will involve “marking” the solutions to some of your peers' previous assignments (you will be assigned new classmates to assess each time). Marking will be done using a rubric provided to you on the Funda site. Marking your peers' work will be assessed and *will* contribute towards your final mark, so make sure you do it.

In other words, the assignment solution you submit will be marked and will contribute towards your final mark. In addition to this, *your marking* of a peer’s solution will *also* be marked and will *also* contribute towards your final mark.

Your marking of a peer’s assignment will be compared with the officially marked one. The more accurate your marking², the higher **your assessment mark** will be.

It is recommended that the structure of your solution has a single project per week, with a single module per task. Use proper naming conventions³.

Part of the assessment of your assignments *will* be on the presentation of your code. So, use proper/descriptive variable names, provide comments, document your methods, format your code appropriately⁴, etc.

² i.e., did you mark correctly, just thumb sucked a mark, just gave high marks, etc.

³ Use these naming conventions (<http://www.oracle.com/technetwork/java/codeconventions-135099.html>).

⁴ This is a soft skill you need to learn for the workplace one day.

Note that you must use Java for all coding done in this module, not Kotlin!

Please also note that if you do not self-assess your submission, you may not query any mark that you receive.

Assignment schedule

The schedule of assignment submissions and peer assessments is given below:

Week & Date		Lecture: Monday 10:25-11:35 05 00 03	Practical: Wed/Thurs 15:45-16:55
1	13-17 Jul	No lectures	
2	20-24 Jul	Course Administration Lecture 1: Introduction to Android	
3	27-31 Jul	Lecture 2: Android views & activities	Start Assignment 1
4	03-07 Aug	Lecture 3: Android persistence (files & databases) & exceptions	
5	10-14 Aug	Lecture 4: Android multiple views (Self-study)	Submit & Peer Assess Assignment 1 Start Assignment 2 Mon 10 Aug Public Holiday
6	17-21 Aug	Lecture 5: Android fragments	Test 1: Saturday, 22 August
7	24-26 Aug	Lecture 6: Collections	Submit & Peer Assess Assignment 2 Start Assignment 3 Lectures end Wed 26 Aug
RECESS 27 August - 02 September			
8	03-04 Sep	RECESS	Lectures begin Thur 03 Sept
9	07-11 Sep	Lecture 7: Multi-threading	Submit & Peer Assess Assignment 3 Start Assignment 4
10	14-18 Sep	Lecture 8: Threads & animations	
11	21-25 Sep	Lecture 9: Networking I	Submit & Peer Assess Assignment 4 Start Assignment 5 Thurs 24 Sept Heritage Day Fri 25 Sept University Holiday
12	28 Sep - 02 Oct	Lecture 10: Networking II	Test 2: Saturday, 03 October
13	05-09 Oct	Lecture 11: Multi-media	Submit & Peer Assess Assignment 5
14	12-16 Oct		Sick assessment: Saturday, 17 October
15	19-21 Oct		
EXAM (examinable modules only) 22 Oct – 13 Nov Recess 14 Nov – 29 Nov RE-EXAM (examinable modules only) 30 Nov – 08 Dec			

Assignment 1

Topics

- Introductory Android
- Activities

Task 1: Android “Hello World”

Follow the video for Lecture 01 (to be shared by your WRPV302 lecturer).

Be sure to deploy it to the emulator and test it. If you have an Android phone or tablet, try deploying to it as well.

Task 2: Android Sequence-Dice

Task Instructions:

For this task, you may only use the Qwen web chat interface (<https://qwen.ai/>) as an AI coding assistant. Qwen (also known as Tongyi Qianwen) is a large language model (LLM) developed by the Alibaba Group.

*Use **Qwen3.7-Max** unless it is unavailable to you. If you use a different Qwen model, that is acceptable, but record the exact name and version in your journal and on your declaration form. You can usually select the model in the top-left of the main chat interface.*

Please read these instructions carefully before attempting this task.

Guidance on LLM Best Practices:

While you will not use Google Gemini LLM in this assignment, there are some best practices on effective prompting and LLM usage as an AI coding assistant that can be found here on the Android Developer documentation:

<https://developer.android.com/studio/gemini/best-practices>

Please also follow the following responsible AI use guidelines as discussed in class:

- Let it help, not do it for you
 - Do not let it generate full classes or complete entire tasks for you
 - Use it wisely because of what it uses in terms of resources to generate a response
 - Balance the speed in getting things done and the struggle to figure out your solution – it is in that struggle where learning happens
- Assessments will be AI-free
 - You will only have access to your cheat sheet during WRPV302 assessments (tests and exams)
- Own every single line of code
 - Academic integrity policies still apply
 - If the developed solution for the assignment/task does not support the required functionality, then “AI did it” is not a valid response
 - Keep in mind that hallucinations can happen, i.e., it can provide you with a confidently incorrect response

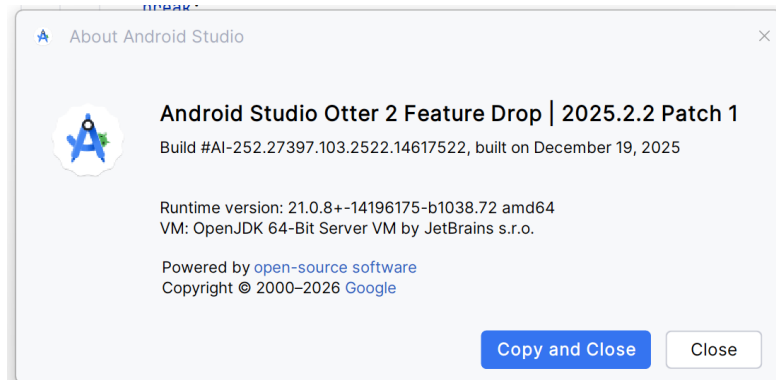
- Respect the module's constraints, e.g., no Kotlin (yet)
 - Follow the rules of the assignment/task and guide the AI tool accordingly
- Don't use it for everything
 - The rubber duck method still works
- Verify, understand, compile, run & test/trace your work

Task Rules Regarding Using AI as a Coding Assistant:

- You must use the LLM in some way as an AI coding assistant.
- You may only use the chat interface and the LLM specified for this task.

Submission Requirements:

- You must include a screenshot of the Android Studio version that you used to create the solution for this assignment/task by taking a screenshot of the window that displays when navigating to Help → About in Android Studio. The image should look like:



- You must complete and upload a *Declaration of AI Tool Assistance* document before being able to access the Funda activity to submit your solution to this assignment.
- You must complete and include an *AI Use Reflection and Journal* document that has links to your AI chats using an AI journal, as well as a reflection on your use, along with your submission for this assignment as a separate document. Please see the document for what needs to be recorded and completed. This will be useful practice for your third-year project as well as in your Honours project, should you continue to CS&IS postgraduate studies.
- You need to submit a zip file of your solution created in the same version of Android Studio as specified for the module, as per the instructions in the *General assignment procedure* section of this document.

Please note that the above instruction sections may change for any subsequent assignments, so please read this section each time.

Task Information:

Sequence-Dice is a variation of the 2-4 player board game called Sequence.

The game is played on a 6x6 board with numbered cells ranging from 2 to 12, two six-sided dice, and coloured tokens representing each player (when playing with four players, there are two teams of two players each).



The *aim* of the game is for a player (or team of players) to complete a *sequence* of five tokens of their colour (or six tokens in a 3 or 4 player game), in either a *connected line* in a row, column or diagonal. The gameplay rules are simple. To decide who *starts* the game, each player rolls the two dice and the player with the highest total starts.

Thereafter, each player takes a turn in clockwise order until a winner is declared.

On a player's turn, they roll the die. The sum of the dice indicates what the player can do:

1. If there is one or more cells with the same number as the sum of the dice that are *empty*, the player must place one of their tokens in one of the corresponding cells.
2. If there are no empty cells with the same number as the sum of the die (including 2 and 12), then the player must *replace* an opponent's token on one of these cells with their own token. If all the cells are occupied by the player's (or team's) tokens, then the player cannot place a token;
3. If a **2** or **12** is rolled, then in addition to #1 and #2 above, the player gets an extra turn;
4. If a **10** is rolled (called a *defensive roll*), then the player must remove one of their opponent's tokens from the board, *except* those in the grey cells (2s and 12s);
5. If an **11** is rolled (called a *wild roll*), then the player must place one of their tokens in any empty cell on the board.

You may watch some videos on YouTube [here](#) and [here](#) to get a feel of how gameplay works.

Note: You will extend this assignment later in the semester into a networked game, so put effort into getting this one working correctly.

You are required to:

1. create a **Model** of the game, called **SequenceDice**:
 - that applies SOLID programming principles⁵;
 - that is Java-only, and has *absolutely* no references to user interfaces (either JavaFX, Android, the console, etc.). This model should represent the *data* of the game (board, number of players, whose turn it is, etc.) and the *game logic* (the rules);
 - that provides methods to query the data, as well as perform certain actions (as described above). If done correctly, you should not need to rewrite this code *at all* in the later assignment; and
 - that is *observable*⁶ (as you did in Assignment 2, Task 3 in WRPV301), so that a listener can receive notifications about interesting things happening with the model [such as the game has started, a player's turn has started (and whose turn it is), the die have been rolled (and what the values and total are), a token has been placed (whose and where?), a token has been removed (whose and where?), the game is over (who won?), etc.]
2. demonstrate that the model is not coupled to a UI by creating a *Java console menu app* that allows the game to be played; and
3. write an Android app that:
 - uses the *exact* same model created above;
 - will allow two to four players to play the game on the *same* device in a *pass-and-play* manner, with turns alternating between the players;
 - visually displays the board, the numbers in each cell, the current die values, the current player and whose tokens have been placed in a cell. To do this, you may simply colour a cell in the player's colour. It is *not* necessary to display images for this assignment, although you may if you wish to. Use whatever Views and interaction methods you think are most appropriate to implement the game. Be sure to let players know when they need to pass the device to the other player so that they may take their turn; and
 - *may* have multiple Activities, but is not required to.

You can change the text colour for a button, label etc. using

```
TextView lbl = findViewById(R.id.lblTest);
lbl.setTextColor(Color.RED);

Button btn = findViewById(R.id.btnTest);
btn.setTextColor(Color.GREEN);
```

The Color class has a number of predefined colours, such as RED and GREEN, that you can use, or you can specify your own colour.

You need an efficient way to determine the row and column of a cell's View when it is clicked on (and for other purposes). One approach is to name the cells so you can iterate through them in a loop (or loops), generate a string name and then look up the

⁵ If you cannot remember the SOLID principles discussed last semester, you can read up about them [here](#).

⁶ If you cannot remember how the observer pattern works, you may read these articles [here](#) and [here](#).

control. The code below demonstrates how to locate a TextView with the id "txtMessage" using a string, instead of the integer R.id.txtMessage

```
int resourceId = this.getResources()  
    .getIdentifier("txtMessage", "id", this.getPackageName());  
TextView txtMessage = findViewById(resourceId);
```

Rubric

Below is the rubric, available on Funda, that will be used when assessing this Assignment:

Criteria	Marks	Option
Task 1: Android "Hello World" In this task, you were expected to follow the lecture demo video and create a running app.	0	Not done.
	1	Done.
Task 2: Sequence-Dice Game (Model) In this task, you were expected to implement a model for the Sequence-Dice game, specifically: 1. Java-only code is used; i.e., no references to UI, System.out, etc. 2. Stores and manages the <i>data</i> required to play the game; 3. Implements the <i>game logic</i> of the game; 4. Provides methods for interacting with the game, e.g., starting a game, placing a token, etc.; and 5. The model should be <i>observable</i>; i.e., other objects can listen for interesting events.	0	Not done, or does not compile.
	2	1-4 of the functionality mentioned is implemented.
	4	All the functionality was implemented, including being observable.
Task 2: Sequence-Dice Game (Console) Using the model, a Java console-menu app was to be created to demonstrate that the model works. The model MUST NOT be altered in any way from the original implementation.	0	Not done, does not compile, or the model was not used correctly.
	2	Uses the model and works correctly.
Task 2: Sequence-Dice Game (Android) Using the model, an Android version of the game was to be created. The game: 1. Should always display sufficient information to see what is happening; 2. Should display the contents of the game board, clearly indicating which tokens are placed where; and 3. Appropriate interaction techniques should be used for placing and removing tokens. The model MUST NOT be altered in any way from the original implementation.	0	Not done, does not compile, or the model was not used correctly.
	2	Basic functionality is present, but some issues.
	4	All the functionality was implemented, and it works correctly.
Task 2: AI Use Reflection and Journal For Task 2, you were required to complete and submit an <i>AI Use Reflection and Journal</i> document. The document should be complete and comprehensive, and it should reflect how you used the specified AI tool.	0	Document is missing or incomplete. OR, the student did not use the specified LLM for the task (e.g., Qwen, you do not need to check the model here; e.g., 3.7-Max).
	2	The document is somewhat complete, demonstrating some usage and reflection on using the LLM as an AI coding assistant.
	4	The document is complete and comprehensive, and reflects how the LLM was used to complete the task.
Android Studio Version Screenshot As part of this assignment, you were required to upload a screenshot of the Android Studio version used to complete it.	0	Not included or does not match the version installed in the CS labs.
	1	Provided and matches the version installed in the CS labs.